# EHRWalk: Synthetic Patient Trajectory Generation Using Heterogeneous Graph Random Walks

Caleb A. Fernandes

***Abstract*—Electronic health records (EHRs) to clinical research, but their use is limited by privacy constraints. Current works utilize LLM based methods which are often slow and cannot capture implicit constraints on the data. Our work introduces EHRWalk, a random-walk–based synthetic data generator that operates on a heterogeneous patient graph. We aim to address the time complexity of generating synthetic data while encoding the constraints inherit to the data through the construction of our graph. We evaluate both directed and undirected graph formulations and conduct a comprehensive ablation study over walk hyperparameters. Our results demonstrate the potential of using random walks to generate synthetic data as well as the limitations.**

***Index Terms*—Electronic Health Records, Synthetic Data, Random Walks, Graph Mining**

## I. Introduction

Electronic health records (EHRs) contain rich information about patient encounters, diagnoses, medications, and provider interactions. Although it enables powerful clinical analysis, real EHR data is rarely accessible due to strict privacy regulations, institutional restrictions, and the risk of re-identification [1]. As a result, synthetic EHR generation has emerged as a promising strategy for enabling research without exposing sensitive information. However, generating synthetic EHRs that preserve statistical and relational structure remains a fundamentally challenging problem.

Unlike conventional tabular datasets, EHRs are inherently *heterogeneous*, *multi-relational*, and *sparsely populated*. A single patient encounter may include multiple diagnoses, multiple medications, multiple providers, and links to temporal histories spanning years. Many features are multi-valued, hierarchical, or represented across multiple tables rather than as single-row attributes. Relationships among entities—such as visit–drug, visit–condition, condition–provider, or person–visit edges—form a complex relational graph rather than an independent-and-identically-distributed (i.i.d.) table. These properties make it difficult for traditional generative models to learn.

Existing state-of-the-art synthetic tabular data generators rely heavily on large generative models such as transformers or diffusion models. While these models achieve strong performance on simpler, single-table benchmarks, they exhibit several limitations when applied to EHR systems. First, most do not naturally support multi-table schemas or many-to-many relationships. Second, they struggle to represent multi-valued cells (e.g., a visit with six diagnoses and three drugs). Third, they introduce substantial computational overhead, often requiring multi-hour training cycles and GPU resources. Finally, they may fail to understand the implicit relationships between features of the dataset. These constraints make them poorly suited for large-scale clinical datasets and for research settings requiring rapid experimentation.

To address these limitations, we propose **EHRWalk**, a graph-based synthetic data generator that leverages random walks on a heterogeneous EHR graph to efficiently produce realistic synthetic patient encounters. By representing the EHR as a multi-relational graph and sampling walks rooted at real visit nodes, EHRWalk naturally captures structural dependencies across tables, supports multi-valued features, and maintains realistic co-occurrence patterns without requiring deep generative models or expensive training. Furthermore, EHRWalk introduces an ablation framework enabling systematic exploration of walk parameters, encounter policies, restart probabilities, and stopping rules, allowing researchers to examine how different graph traversal strategies affect downstream synthetic data quality. Our method aims to provide a scalable and flexible alternative to existing generative approaches, preserving essential relational structure while significantly reducing computational cost.

## II. Related Work

### A. LLM- and Diffusion-Based Tabular Data Synthesis

Current methods involve generative modeling for tabular data using large language models (LLMs) or diffusion processes. GReaT [1] demonstrates that autoregressive transformers can generate realistic synthetic tables by linearizing rows into token sequences and sampling conditionally on observed subsets of columns. Despite strong performance on heterogeneous tabular benchmarks, GReaT assumes a single flat table and does not natively support multi-valued fields or multi-table schemas. GReaTER [2] extends this framework through implementing a multi-table schema, but requires manual encoding of dependencies and turning ambiguous features such as IDs into meaningful tokens for the LLM. Where GReaTER fails is that it is less effective in domains where features are highly correlated, since it classifies and separates by independence.

Diffusion-based methods provide another successful class of tabular generators. TabDDPM [3] and TabSyn [4] are both diffusion models that work by adding noise to real data and then learning how to reverse that process. After training, the model starts at pure noise will "denoise" it, which creates synthetic records. This approach works very well for simple tables where every row is just a flat list of features, but requires intensive feature engineering to frame EHR data into that format.

### B. Random-Walk-Based Synthetic Graph Generation

Beyond row-based models, CTWALK[1] [5] introduces a hybrid generative framework for temporal networks with node and edge attributes. The method combines TAGGEN, which learns distributions over temporal random walks, with a conditional tabular GAN (CTGAN) that captures time-varying attribute dependencies along these walks. CTWALK successfully preserves edge weight patterns, node labels, and temporal evolution in time-varying networks. However, the architecture is computationally heavy and is tailored for graph time series. This may prove useful for formulating the journey of a patient in EHR data, but would require meticulous feature engineering.

### C. Privacy-Oriented Synthetic Data with Random Walks

A separate line of work studies theoretical guarantees for synthetic data generation under formal privacy constraints. Boedihardjo et al. [6] develop a metric-privacy framework and a polynomial-time algorithm for constructing *private measures* over compact metric spaces, from which synthetic samples can be drawn. A key innovation is a new superregular random walk, whose joint distribution of steps is as regular as that of independent random variables, yet which deviates from the origin. This work may prove useful for handling the privacy component of EHR data which we do not address in EHRWalk.

## III. Data and Graph Construction

EHRWalk operates on a heterogeneous graph representation of EHR, requiring careful preprocessing and integration of multiple OMOP-formatted tables. We extract four primary datasets from the provided EHR sample: `sampled_person`, `sampled_condition_occurrence`, `sampled_drug_exposure`, and `provider`. These tables collectively describe patients, clinical encounters, diagnoses, medication exposures, and provider characteristics. Before constructing the graph, we apply a standardized preprocessing pipeline to ensure data consistency and semantic coherence across the heterogeneous structure.

### A. Data Cleaning and Normalization

All core identifier fields—including *person_id*, *visit_occurrence_id*, *provider_id*, *drug_concept_id*, and *condition_concept_id*—are coerced to numeric types and malformed records removed. Duplicate rows are dropped, implausible entries such as missing IDs are filtered, and each table is reindexed for consistency. We further compute descriptive statistics for each feature category, including degree distributions for drugs, conditions, providers, persons, and visits. These distributions consistently exhibit heavy-tailed behavior as seen in Figure 2, with many rare concepts appearing only once or twice in the dataset. Preserving this empirical topology is essential, as the graph-based sampling performed by EHRWalk is sensitive to node degree patterns and relies on these structures to generate realistic synthetic observations.

[1]This work has not been published, though we may find it useful. It is available on OpenReview: https://openreview.net/pdf?id=U9fSUihGP0Q

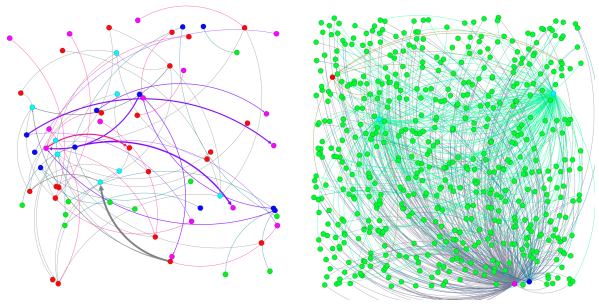


Fig. 1: Subsampled heterogeneous EHR graph. Directed (left), undirected (middle), and node-type legend (right). Visit (green), Drug (light blue), Provider (red), Condition (pink), Person (blue). *Edge color does not represent anything*

### B. Graph Schema and Node Types

The cleaned tables are then merged to form a heterogeneous graph containing five core node types:

- **Visit nodes**, representing individual clinical encounters.
- **Condition nodes**, representing diagnosis concepts linked to visits.
- **Drug nodes**, representing medication exposures associated with visits.
- **Provider nodes**, representing care providers with their respective specialty information.
- **Person nodes**, representing the patient associated with each visit.

Each node stores both (i) a raw concept identifier and (ii) a node type indicating which field of the synthetic record it is allowed to populate during the generation. This mapping ensures that walks produce syntactically valid synthetic observations and prevents incompatible node types from filling incorrect fields.

### C. Edge Construction

Edges encode the relational structure of the EHR and serve as the backbone for random walk traversal. We construct two graph variants: directed and undirected 1. Directed edges are constructed following a clinically plausible ordering: *VisitID* $\rightarrow$ *Patient* $\rightarrow$ *Condition* $\rightarrow$ *Provider* $\rightarrow$ *Drugs*. Undirected edges are formed as the Visit nodes become anchors that the features belonging to that visit are attached to. If a feature appears in multiple visits, then each visit is connected to that feature. In both directed and undirected graphs, each unique value in the table has one node representation in the heterogeneous graph, resulting in the same node count for each node, agnostic to the graph type. However, both graphs do not contain identical edge distributions as seen in Table 1[2].

[2]In counting the degree of nodes in the directed graph, the degree is the sum of both the in-degree and out-degree

### D. Handling Multi-Valued Clinical Features

A major challenge in EHR data integration is that visits often contain multiple conditions and drug exposures. Instead of flattening these into multi-valued columns—as required by many tabular generative models-we represent each as a separate node with edges to the corresponding visit. This graph-based representation naturally supports many-to-many relationships and accommodates heterogeneity.

TABLE I: Node-type statistics for the undirected and directed heterogeneous EHR graphs. Edge participation denotes the percentage of edges associated with each node type.

| Node Type | Count | Undirected | | Directed | |
|---|---|---|---|---|---|
| | | Avg Deg | Edge Part. | Avg Deg | Edge Part. |
| visit | 43,559 | 9.06 | 50.00% | 1.00 | 7.36% |
| provider | 6,809 | 6.40 | 5.52% | 30.28 | 34.83% |
| condition | 4,594 | 27.45 | 15.97% | 26.87 | 20.85% |
| drug | 3,893 | 46.61 | 22.99% | 33.13 | 21.79% |
| person | 1,712 | 25.44 | 5.52% | 52.44 | 15.17% |
| **Total Nodes** | 60,567 | — | | — | |
| **Total Edges** | — | 394,648 | | 295,945 | |

## IV. EHRWalk Methodology

EHRWalk generates synthetic patient observations by performing restart-based random walks on a heterogeneous EHR graph. The method requires no training and captures the multi-relational structure of clinical data directly from the graph topology.

### A. Overview

Given a heterogeneous graph $G = (V, E)$, a synthetic sample is generated by starting a walk at a real visit node $v_0$ and filling a predefined set of column types $\mathcal{C} = \{c_1, \ldots, c_K\}$. Each node $u \in V$ stores a raw identifier and a column type label $\tau(u) \in \mathcal{C}$, ensuring only compatible nodes populate the corresponding fields. A synthetic record $x$ initially stores all fields as unfilled.

### B. Walk Initialization

The walker begins at $u_0 = v_0$. At time $t$, the walk state is $(u_t, x_t)$, where $x_t$ is the partially constructed record. If $\tau(u_t) = c_i$ and $x[c_i]$ is unfilled, the identifier of $u_t$ is recorded. One value for each column type is recorded per synthetic row, with the exception of conditions and drugs. There can be multiple conditions and drugs prescribed during a patient visit, but only one provider, patient, and visit.

### C. Transition Distribution

Let $N(u)$ denote the neighbors of node $u$ and let $w_{uv}$ be the edge weight. At each step, the walk returns to the originating visit with probability $p$. This prevents the walk from drifting into unrelated portions of the EHR graph and ensures that generated sequences stay anchored to the local clinical context of the patient. Several transition distributions are supported.

- **Uniform transition** Each neighbor is chosen with equal likelihood. This provides an unbiased walk that treats all clinical relationships as equally informative, useful as a baseline without structural preference.
- **Edge-weighted transition** Neighbors connected by stronger (higher-weight) edges are preferred. In EHR graphs, heavier edges represent more frequent or clinically meaningful co-occurrences (e.g., common drug–condition pairs), so this weighting guides the walk toward relationships that appear more often in real data.
- **Inverse-degree transition** Neighbors that are highly connected (e.g., very common diagnoses) are down-weighted, while more specific nodes are favored. This reduces drift into generic hubs in the EHR (such as "hypertension" or "follow-up visit") and helps the walk remain within more informative, patient-specific regions of the graph.

**Algorithm 1** EHRWalk Synthetic Record Generation

**Require:** Starting visit node $v_0$, required column types $\mathcal{C}$, restart probability $p$, maximum walk length $T$, transition rule, encounter policy, stopping rule

Create an empty synthetic record $x$ with one slot for each column type in $\mathcal{C}$
Set the current node $u$ to the starting visit $v_0$
**for** $t = 1$ to $T$ **do**
    If $u$ corresponds to a column type we have not yet filled in, record its identifier in $x$
    If all required columns are now filled, return the completed record
    Draw a random number to decide the next move
    **if** we trigger a restart (with probability $p$) **then**
        Jump back to the starting visit $v_0$
    **else**
        Move to one of $u$'s neighbors, using the chosen transition rule
    **end if**
**end for**
Return the (possibly partially filled) synthetic record $x$

### D. Encounter Policies

Two strategies control when the walker records a feature:

- **First-encounter**: record the first node of type $c_i$ encountered.
- **Second-encounter**: skip the first node of that type and record the second distinct one.

This allows control over variability and proximity to the root visit, introducing more variability into the walk.

### E. Stopping Rules

Two termination rules are provided.

- **Complete Coverage**: record the first node of type $c_i$ encountered.
- **Percent Coverage**: stop when $\alpha$ percent of the synthetic columns are filled.

## V. Ablation Framework

EHRWalk exposes several knobs that control how random walks traverse the heterogeneous EHR graph and how synthetic records are constructed. To understand the impact of these design choices, we conduct an ablation study that sweeps a grid of hyperparameters and evaluates each configuration using the metrics described in Section VI. For every setting, we generate a full synthetic dataset, run the evaluation pipeline, and log both the knob values and scalar metrics into a single summary file (`ablation_results.csv`).

### A. Parameters Explored

The ablation varies seven primary parameters:

- **Graph type** (`graph_label`): we compare random walks on the undirected heterogeneous graph (`"un"`) and the directed graph (`"dir"`). The former emphasizes a symmetric co-occurrence structure, while the latter replicates the patient's journey.
- **Restart probability** (`restart_prob`): the probability $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ of jumping back to the root visit node at each step. Higher values of $p$ keep walks tightly localized around the originating encounter, whereas lower values allow exploration of more distant regions of the graph.
- **Edge weighting** (`use_edge_weight`): a Boolean flag indicating whether transitions use raw edge weights (e.g., frequency-based weights) or treat all edges as uniform. When enabled, transition probabilities are proportional to $w_{uv}$ as described in Section III.
- **Inverse-degree reweighting** (`inverse_degree`): a Boolean flag that, when enabled, downweights transitions into high-degree nodes by multiplying $w_{uv}$ by $1/\deg(v)$ before normalization. It reduces the chance of walking into hub nodes, which have a higher probability of connecting unrelated events.
- **Encounter policy** (`encounter_policy`): either `"first"` or `"second"`, controlling whether the first or second distinct encounter with a given column type is recorded when filling the synthetic record.
- **Stopping rule** (`stop_rule`): either `"complete"` or `"percentage"`. Under the complete rule, the walk continues until all required fields are filled or the maximum number of steps is reached. Under the percentage rule, the walk stops once a fixed fraction of fields has been filled.
- **Maximum steps** (`max_steps`): a hard cap on the number of walk steps per visit, explored over $\{100, 200, 300\}$. This provides a trade-off between coverage and runtime, and prevents excessive wandering in sparse regions of the graph.

### B. Experimental Grid

We construct a full factorial grid over the parameter options:

- `graph_label` $\in \{\text{un}, \text{dir}\}$,
- `restart_prob` $\in \{0.1, 0.2, 0.3, 0.4, 0.5\}$,
- `use_edge_weight` $\in \{\text{True}, \text{False}\}$,

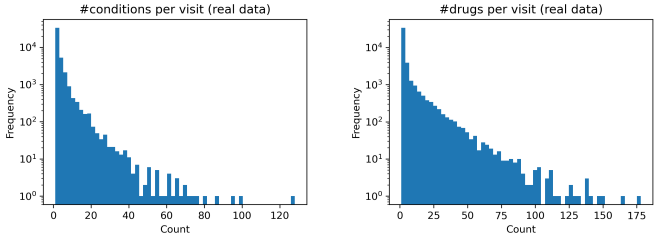



Fig. 2: Distribution of Conditions (left) and Drugs (right) per visit in the REAL data

- `inverse_degree` $\in \{\text{False}, \text{True}\}$,
- `encounter_policy` $\in \{\text{First}, \text{Second}\}$,
- `stop_rule` $\in \{\text{Complete}, \text{Percentage}\}$,
- `max_steps` $\in \{100, 200, 300\}$,

with `stop_percentage` fixed at $0.8$ whenever the percentage rule is used. This yields a total of $480$ ablation configurations.

For each configuration, we select the corresponding graph (directed or undirected), call the random walk generator to produce a synthetic visit-level dataset, and then run the full evaluation pipeline:

1) Wasserstein distance on the number of drugs and conditions per visit,
2) $P$-test (logistic regression) for REAL vs SYNTH classification,
3) Provider-specialty logistic regression under TRTR, TSTR, and TR+S→R regimes.

The resulting scalar metrics and hyperparameters are written as a single row in `ablation_results.csv`. This consolidated table forms the basis for all subsequent analysis.

To summarize the ablation study and identify operating points for EHRWalk, we use a selection script (ParamDetect) that reads `ablation_results.csv` and ranks configurations according to different evaluation criteria. For each metric of interest, ParamDetect identifies the configuration that optimizes that metric and records the corresponding hyperparameters. Ties are broken using secondary criteria such as favoring smaller `max_steps` to prioritize more efficient configurations when performance is comparable.

## VI. Evaluation Metrics

We evaluate the quality of synthetic data generated by EHRWalk along three complementary dimensions: (i) distributional similarity of simple visit-level statistics, (ii) real-vs-synthetic discriminability via a $p$-test, and (iii) predictive utility for a downstream provider specialty prediction task under TRTR, TSTR, and TR+S→R regimes.

### A. Distributional Similarity via Wasserstein Distance

To assess whether EHRWalk preserves basic visit-level utilization patterns, we compute the distributions of the number of drugs and the number of conditions per visit for both REAL and SYNTH datasets. From the semicolon-separated lists of drug and condition concept IDs, we derive integer-valued counts

$$n_{\text{drugs}}(v), \quad n_{\text{conds}}(v)$$

for each visit $v$, and obtain empirical distributions $P_{\text{real}}$ and $P_{\text{synth}}$ over these counts.

We then compute the one-dimensional 1-Wasserstein (Earth Mover's) distance between the REAL and SYNTH distributions for both drugs and conditions. Lower values indicate closer alignment of synthetic distributions to real distributions.

### B. P-Test: Real vs Synthetic Classification

The P-test determined if a classifier distinguish REAL data from SYNTH data. For every visit, we build a set of tokens from its drug and condition concept IDs, encode these with a `MultiLabelBinarizer` into sparse multi-hot feature matrices for REAL and SYNTH, and assign labels $y = 1$ (REAL) and $y = 0$ (SYNTH). We then concatenate the two datasets, perform a stratified $80/20$ train–test split, and train a logistic regression classifier. On the held-out test set we report AUC and accuracy. The classifier cannot reliably distinguish REAL from SYNTH when the AUC $\approx 0.5$, the accuracy $\approx 0.5$, and a large $p$-value.

### C. Predictive Utility: Provider Specialty (TRTR / TSTR)

Beyond distributional similarity and discriminability, we evaluate whether synthetic data preserves clinically meaningful structure by measuring performance on a downstream supervised task: predicting provider specialty from multi-hot drug+condition features.

For each visit, we attach a provider specialty via a mapping from `provider.csv`. In REAL data, we count the frequency of each specialty and retain only those with at least `min_class_size` visits; all other labels are mapped to `Other`, and visits with `Unknown` specialty are excluded from the supervised task. The result is a multi-class classification problem over a reduced, clinically meaningful label space.

We reuse the multi-hot feature matrices $X_{\text{real}}$ and $X_{\text{synth}}$ and evaluate three regimes using logistic regression (one-vs-rest, `liblinear` solver, `max_iter`=1000):

- **TRTR (Train Real, Test Real)**: Train on a subset of REAL visits, test on a held-out REAL set. This serves as a baseline.
- **TSTR (Train Synthetic, Test Real)**: Train on SYNTH visits, test on the same REAL test set as TRTR. This measures how well models trained purely on synthetic data transfer to real patients.
- **TR+S→R (Train Real+Synthetic, Test Real)**: Train on the union of REAL training visits and all labeled SYNTH visits, test on REAL. This captures whether adding synthetic data improves over the real-only baseline.

For each regime, we report macro-averaged and micro-averaged F1 scores. Let $F^{\text{macro}}_{\text{TRTR}}$ and $F^{\text{macro}}_{\text{TSTR}}$ denote the macro F1 scores in the TRTR and TSTR settings, respectively, and $F^{\text{macro}}_{\text{TR+S}\to\text{R}}$ the macro F1 for the mixed training regime. We summarize relative utility using ratios

$$u_{\text{TSTR}} = \frac{F^{\text{macro}}_{\text{TSTR}}}{F^{\text{macro}}_{\text{TRTR}}}, \qquad u_{\text{TR+S}\to\text{R}} = \frac{F^{\text{macro}}_{\text{TR+S}\to\text{R}}}{F^{\text{macro}}_{\text{TRTR}}}.$$

Values of $u_{\text{TSTR}}$ close to 1 indicate that synthetic-only training nearly matches real-only training, while $u_{\text{TR+S}\to\text{R}} > 1$ suggests

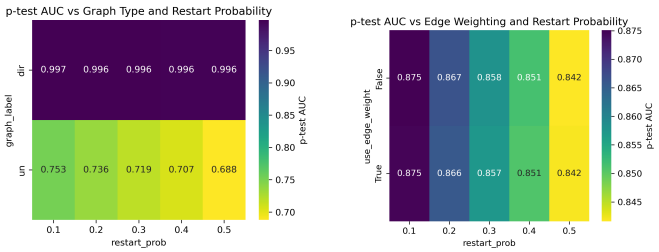



Fig. 3: P-test Heatmaps for Restart Probability vs Graph Type (left) and Restart Probability vs Edge Weighting (right).

that augmenting real data with synthetic examples can improve downstream performance. In addition, we compute per-class F1 scores to inspect which specialties are best preserved by EHRWalk.

## VII. Experiments

### A. Experimental Setup

All experiments are conducted on a single OMOP-derived EHR sample, which contains visit-level information for persons, drug exposures, condition occurrences, and provider identifiers. After preprocessing and normalization (Section II), we construct both the undirected and directed heterogeneous graphs that serve as substrates for EHRWalk. Each real visit node in the graph produces exactly one synthetic record per configuration in the ablation grid, ensuring a one-to-one correspondence between real and synthetic visit counts.

Our primary experimental objective is to evaluate how different random-walk design choices affect the realism and utility of the generated synthetic datasets. To this end, we apply the full ablation grid described in Section VI, consisting of 480 distinct configurations spanning graph types, walk dynamics (restart probability, edge weighting, inverse-degree weighting), encounter policies, stopping rules, and maximum walk lengths. For each configuration, we generate a complete synthetic dataset and evaluate it using the metrics introduced in Section V: Wasserstein distance on visit-level multiplicities, $p$-test discriminability, and predictive utility for provider specialty classification.

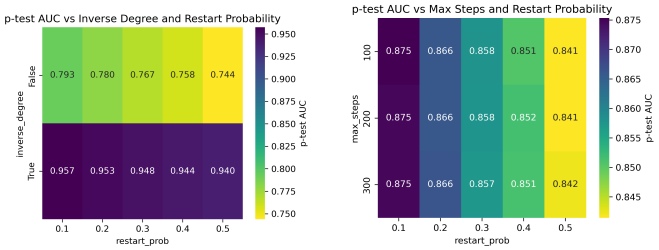



Fig. 4: P-test Heatmaps for Restart Probability vs Inverse Degree (left) and Restart Probability vs Max Steps (right).

### B. Implementation Details

Synthetic datasets are generated using the `rw_ablation` script, which executes parallel random walks over the heterogeneous graph. We use 28 worker processes and a fixed random seed for reproducibility. Each walk begins at the

corresponding real visit node and proceeds according to the selected transition distribution, encounter policy, stopping rule, and maximum step constraint.

After generation, each synthetic dataset is processed before evaluation. Lists of drug and condition IDs are parsed into multi-hot vectors using a single union vocabulary across REAL and SYNTH. Provider specialties are attached using `provider.csv`. The synthetic datasets are then fed into the evaluation framework where they are tested across the methods stated in Section VI, and their metrics are recorded into one aggregate table.

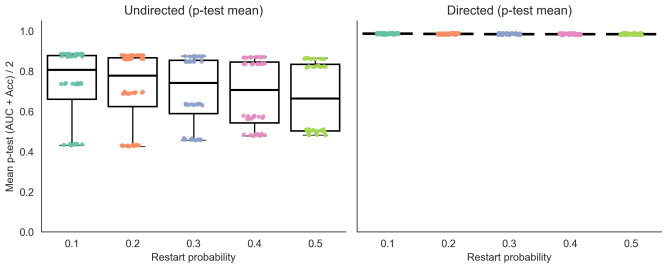



Fig. 5: Distributions of P-test values across restart probabilities for undirected and directed graphs

### C. Visualizations

We computed standard visualizations on the REAL data distributions prior to synthetic generation to understand irregularities that may be present in the data and use this information to shape the choices of our model. Five plots were developed, two of which can be seen in Figure 2 the rest can be seen in our GitHub.

To analyze our 480 ablations, we computed heatmaps across the hyperparameters in relation to our evaluation metrics, distributions as seen in Figures 3, 4, 7, 8, with the rest of the heatmaps in our GitHub. Upon analyzing these heatmaps, we were motivated to develop more plots to describe our results as seen in Figures 6, 5.

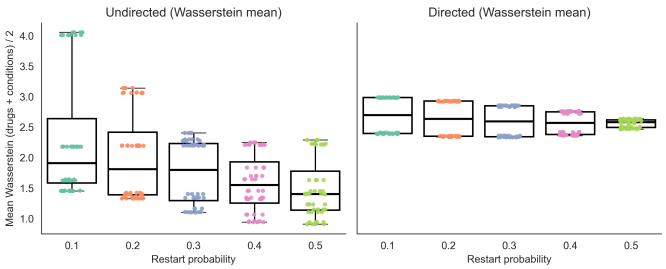



Fig. 6: Distributions of Wasserstein distances across restart probabilities for undirected and directed graphs

### D. Baseline Comparison

We next compare EHRWalk against GReaT, a transformer-based tabular synthetic data generator, using the same evaluation metrics and a matched evaluation subset. GReaT is evaluated on 5,000 synthetic visits generated with the `tabularisai/Qwen3-0.3B-distil` backbone, while EHRWalk is evaluated on 5,000 synthetic visits with these parameters:

- **Graph:** Undirected
- **Restart Prob:** 0.3
- **Edge Weight:** True
- **Inverse Weighting:** False
- **Encounter Policy:** First
- **Stop Rule:** Completion
- **Max Steps:** 300

Figure 9 illustrates the resulting visit-level distributions for EHR alongside the REAL data. We did not provide the distribution for GReaT because its distribution was nonexistent, which will be discussed in Section VIII.

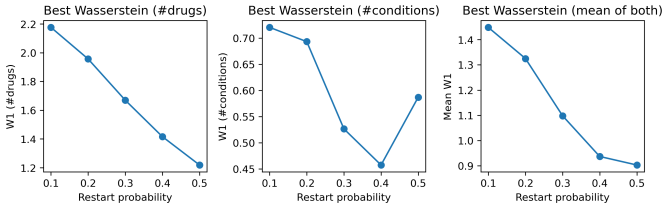



Fig. 7: Best 1-Wasserstein distance for the drugs (left) and conditions (center) per visit, and their mean (right), as a function of restart probability.

## VIII. Results

### A. Real Distributions

Figure 2 summarizes the empirical distributions of the number of drugs and conditions per visit in the REAL dataset. These variables exhibit pronounced heavy-tailed behavior: while the average visit contains only a small number of drugs and diagnoses, a nontrivial fraction of encounters accumulates dozens of codes. For example, in our sample of 43,559 visits the mean number of drugs per visit is approximately 4 and the mean number of conditions is roughly 3, but the maximum values exceed 170 drugs and 120 conditions on a single visit.

Such scale-free patterns motivated several aspects of the EHRWalk design. First, we model drugs and conditions as separate node types, and allow multiple instances of each to be collected along a single random walk rather than constraining walks to exactly one drug and one condition per visit. Second, the presence of generic, high-degree nodes (e.g., demographic attributes or very common diagnoses) highlighted the risk that hub nodes could connect otherwise unrelated visits, potentially destroying the implicit dependence structure encoded in the graph. This concern motivated our exploration of inverse-degree weighting and related ablations discussed below.

### B. Ablation Outcomes

Because EHRWalk exposes several interacting hyperparameters, no single configuration simultaneously optimizes all metrics. In particular, the restart probability trades off fidelity against novelty: high restart probabilities keep walks tightly anchored to the originating visit—effectively copying local neighborhoods—while low restart probabilities encourage exploration at the cost of increased distributional drift. Rather than selecting a single "best" configuration, we therefore analyze trends across the ablation grid and provide a separate

`ParamDetect` script to identify the top configuration for each metric at each restart probability.

Figure 3 reports the mean of AUC and Accuracy for p-test for REAL vs. SYNTH classification across restart probabilities and graph types. Across the full grid, the undirected graph variant consistently produces lower p-test values than the directed variant, indicating that synthetic data generated on the undirected graph is harder for the classifier to distinguish from REAL data. These trends are further illustrated by the boxplots in Figure 5, which show the distribution of the mean p-test score (average of AUC and accuracy) for each restart probability, separately for undirected and directed graphs. For the undirected graph, the distributions have values reaching $0.5$ (near indistinguishable), while for the directed graph the values are clustered near $0.99$ with no variance. Together, these results suggest that encoding strict temporal directionality in the current graph construction is detrimental for realism, and that the undirected co-occurrence structure is the more appropriate substrate for EHRWalk in its present form. We will expand more on this in Section IX, because, intuitively, we want the directed graph to perform better. This way are able to generate the patient's journey.

Figure 3 compares edge-weighted vs. unweighted transitions. Across restart probabilities, the difference in p-test between edge weights and uniform transitions is negligible. This indicates that, for this EHR graph, simply following adjacency structure is sufficient to recover realistic co-occurrence patterns and that explicitly favoring more frequent edges does not meaningfully improve (or harm) realism.

The two more ablations are summarized in Figure 4. Inverse-degree weighting consistently harms realism: configurations with inverse-degree enabled produce substantially higher p-test values than their counterparts. Intuitively, penalizing high-degree nodes over-corrects for hub bias in this dataset and steers walks toward rare concepts that are poorly supported in the REAL data, yielding synthetic visits that are easy to classify as fake. In contrast, the maximum step budget has little systematic effect on realism. The corresponding heatmaps show minimal variation in p-test as `max_steps` increases (e.g., from 100 to 300), revealing that we can favor runtime by having smaller walk lengths without sacrificing realism.

Figure 7 tracks the best Wasserstein distance for the number of drugs and conditions per visit as a function of restart probability. For drugs, the 1-Wasserstein distance decreases linearly as restart probability increases, reflecting progressively closer alignment between REAL and SYNTH utilization counts when walks remain tightly localized around their root visits. A similar trend holds for conditions up to $p = 0.4$, after which we observe a modest upward jump at $p = 0.5$. Yet, this irregularity is inconsistent with Figure 2 since both condition and drug exhibit the same scale-free distributions.

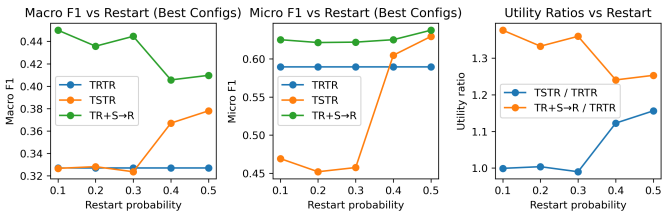



Fig. 8: Macro- and micro-F1 scores for provider-specialty prediction under TRTR, TSTR, and TR+S→R regimes, as a function of restart probability.

Figure 8 demonstrates the downstream utility of EHRWalk samples for provider-specialty prediction. We use the best TRTR macro- and micro-F1 as a baseline against which to compare utility at each restart probability. In both the macro- and micro-F1 plots, TSTR underperforms TRTR for restart probabilities $p \leq 0.3$, but approaches or slightly exceeds TRTR once $p \geq 0.4$, mirroring the improvements in Wasserstein and p-test metrics at higher restart probabilities. More importantly, the TR+S→R regime consistently outperforms TRTR across nearly all restart probabilities, with utility ratios often in the 1.1–1.3 range. This indicates that, even when synthetic-only training is suboptimal, augmenting REAL data with EHRWalk samples can provide a statistically meaningful performance boost.

To better understand these trends, Figure 6 shows boxplots of the mean Wasserstein distance (averaged across drugs and conditions) for each restart probability, separately for undirected and directed graphs. As the restart probability increases, both the mean and variance of the Wasserstein distance decrease, particularly for the undirected case. The directed graph remains consistently poor-performing, with higher mean distances, yet still exhibits a reduction in dispersion at higher restart probabilities. Taken together with Figure 5, these results suggest that while undirected EHRWalk configurations at moderate-to-high restart probabilities produce synthetic data that is both realistic (by p-test) and distributionally aligned (by Wasserstein), lower restart probabilities ($p \leq 0.3$) yield larger distributional gaps. Even though they are realistic, the high variance in distribution makes them unfit for sole training. The key positive outcome is that TR+S→R consistently outperforms TRTR regardless of restart probability, enabling researchers to expand training sets with EHRWalk samples and achieve better performance than using REAL data alone.

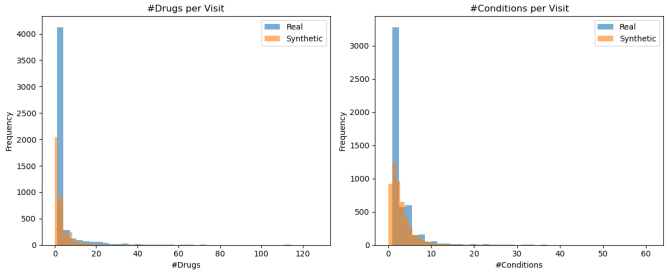



Fig. 9: Distribution of Conditions and Drugs for REAL data, and EHRWalk synthetic data on the 5,000 node subset used for comparison against GReaT.

### C. GReaT vs. EHRWalk

Comparing EHRWalk against GReaT provided meaningful insights. On basic distributional similarity, EHRWalk substantially outperforms GReaT. For GReaT, the Wasserstein distance between REAL and SYNTH counts is $W_{\text{drugs}} = 3.60$ for the number of drugs per visit and $W_{\text{conditions}} = 2.89$ for the number of conditions per visit. In contrast, EHRWalk achieves $W_{\text{drugs}} = 1.57$ and $W_{\text{conditions}} = 0.46$, corresponding to roughly 55–60% and 80–85% reductions, respectively. Figure 9 displays the distributions for EHRWalk. Thus, EHRWalk more accurately preserves the marginal utilization patterns of both medications and diagnoses.

The p-test tells a complementary story. For GReaT, the REAL vs. SYNTH classifier attains a AUC of $0.9963$ and an accuracy of $0.9853$, indicating that the synthetic records are almost perfectly separable from REAL ones. By our criterion—AUC and accuracy close to $0.5$—the GReaT samples are therefore highly unrealistic, despite their apparent tabular plausibility. EHRWalk, by contrast, yields a p-test AUC of $0.3318$ and accuracy of $0.3930$. Both values are much closer to $0.5$ than those of GReaT.

The most striking differences emerge in the downstream provider-specialty task. For GReaT, the TRTR baseline (train on REAL, test on REAL) achieves macro-F1 $= 0.3269$ and micro-F1 $= 0.5894$. When training solely on GReaT synthetic data (TSTR), performance collapses to macro-F1 $= 0.0082$ and micro-F1 $= 0.0414$, yielding a utility ratio TSTR/TRTR $\approx 0.025$. Augmenting REAL data with GReaT samples (TR+S→R) essentially recovers the baseline (macro-F1 $= 0.3224$, utility ratio $\approx 0.99$) but does not offer a meaningful gain.

EHRWalk exhibits the opposite behavior. On the same evaluation subset, the TRTR baseline attains macro-F1 $= 0.2983$ and micro-F1 $= 0.4511$. Training exclusively on EHRWalk synthetic data (TSTR) *improves* performance to macro-F1 $= 0.3523$ and micro-F1 $= 0.5459$, corresponding to a utility ratio TSTR/TRTR $\approx 1.18$. Adding EHRWalk samples to REAL data (TR+S→R) further increases macro-F1 to $0.4412$ (micro-F1 $= 0.5428$), with a utility ratio TR+S $\rightarrow$ R/TRTR $\approx 1.48$. In other words, unlike GReaT, EHRWalk supports both synthetic-only training that outperforms the real-only baseline and mixed training that yields substantial improvements over TRTR. Per-class F1 scores reinforce this picture: GReaT often delivers near-zero TSTR F1 for many specialties, whereas EHRWalk maintains moderate to high F1 scores (often 0.3–0.8) across a broad range of common specialties, indicating that it better preserves clinically meaningful structure in the multi-label feature space.

EHRWalk is also substantially more efficient. In our experiments, approximately $45{,}000$ synthetic visits were generated in under two minutes with parallel random walks, whereas GReaT required roughly $90$ minutes of CPU time to generate $5{,}000$ visits on the same machine (NVIDIA RTX 5090 GPU, AMD Threadripper 7970X CPU). Random-walk cost scales approximately linearly with the number of visits and with the chosen `max_steps`, and walk generation is parallel across visits. Memory usage remains moderate, as the heterogeneous graph is loaded once and shared across workers. Overall, EHRWalk demonstrates favorable scaling characteristics for large EHR datasets, enabling rapid exploration of walk parameters and efficient execution of full ablation studies.

The failure of GReaT could be attributed to its preprocessing. GReaT requires the values in each cell of the table to be singular values, while our data has multiple values for conditions and drugs in a single cell. We cannot preprocess such that there is only one drug or condition per cell as multiple of each can occur in a single visit. Thus, we converted our cells into a semantically meaningful list for GReaT's LLM to interpret. The format of our cells were "['DrugID', 'DrugID', ... , 'DrugID']" and the same was done for conditions.

### D. Overall Performance

Taken together, these results show that a lightweight, graph-based random-walk approach can serve as a simple foundation upon which we can develop adapted synthetic generation techniques. The ablation study identifies undirected EHRWalk configurations with moderate-to-high restart probability as a robust operating regime: they deliver low Wasserstein distances, near-indistinguishable p-test scores, and strong provider-specialty prediction performance, particularly when synthetic data is used to augment REAL training sets. These findings suggest that structurally informed, graph-centric methods like EHRWalk are a promising direction for scalable synthetic EHR generation.

## IX. Limitations

Due to the complexity of EHR data and the simplicity of EHRWalk, EHRWalk has its own limitations. These limitations are described below.

- **High restart probabilities favor memorization.** Configurations with large restart probability effectively remain anchored to the originating REAL visit, making the walk behave like a near-copying mechanism. Provide better Wasserstein metrics as seen in Figure 6, not because they capture deeper structure, but because they reproduce the observed data too closely. The good news is that the p-test in Figure 5 provides evidence that realism is agnostic of restart probability.
- **Current implementation favors undirected graphs.** EHRWalk performs best on undirected graphs, which ignore the temporal ordering inherent in clinical data. Directed variants are desirable for capturing a patient's care trajectory over time, but require further methodological development to handle directed heterogeneity and asymmetric neighborhood structure.
- **Potential privacy concerns.** Provider and patient identifiers are currently preserved, which may expose sensitive information. The current version of EHRWalk does not address privacy concerns.
- **Scalability limitations.** As the number of features grows, the heterogeneous graph can become extremely large and memory-intensive. Practical deployment on full-scale

EHR systems may require graph partitioning, sampling strategies, or distributed construction.

- **Susceptibility to hub-node bias.** Highly connected nodes (e.g., generic demographic attributes like "Male") can dominate random-walk paths and lead to unrealistic co-occurrence patterns. These hubs act as shortcuts across unrelated regions of the graph, reducing clinical specificity in the generated records. Although we showed that inverse weighted does not make a difference, it is important to keep this danger in mind when more general features such are added to the dataset.
- **Extensive preprocessing requirements.** Constructing the heterogeneous graph requires substantial upfront merging, cleaning, and normalization of multiple relational EHR tables. This preprocessing burden may limit adoption and complicate reproducibility.
- **Sensitivity to graph-construction choices.** Results depend heavily on design decisions such as which tables to include, how edges are weighted, and how node types are defined. Small changes in preprocessing or schema yield different graphs and thus different synthetic outputs.
- **Limited modeling of multi-type interactions.** Although the graph is heterogeneous, the random walk treats all edges as a uniform type. More expressive modeling of relation-specific transitions could better capture clinical structure, but is not yet implemented.

## X. Conclusion

EHRWalk introduces a simple yet structurally grounded approach to synthetic EHR generation by leveraging random walks on heterogeneous clinical graphs. Through an extensive ablation study spanning graph structure, restart dynamics, transition rules, and encounter policies, we demonstrate that random walks can recover meaningful clinical co-occurrence patterns and support downstream utility. Undirected variants with moderate-to-high restart probabilities yield synthetic data that is both difficult to distinguish from REAL data (Fig. 5) and well aligned with visit-level utilization distributions (Fig. 6). Moreover, augmenting REAL training data with EHRWalk samples consistently improves provider-specialty prediction performance across nearly all restart probabilities (Fig. 8), highlighting the method's potential as a scalable and flexible generator. Although not the primary focus of this work, the comparison against GReaT underscores EHRWalks potential as a synthetic generator.

Our findings expose several important avenues for future development. The directed graph—despite encoding a clinically intuitive temporal flow—underperforms due to its restrictive topology, suggesting the need for more expressive directed-walk mechanisms or temporal random-walk models that better reflect patient trajectories. Figure 9 shows that distributional alignment still lags REAL data in several regimes, indicating that refined walk strategies or relation-specific transition modeling may further improve fidelity. Privacy remains an open challenge, particularly with respect to patient and provider identifiers, motivating integration with emerging privacy-preserving random-walk frameworks. Taken together, this work provides a foundational platform: a transparent and computationally efficient method that captures essential EHR structure while offering ample headroom for advances in temporal modeling, privacy preservation, and enhanced distributional accuracy.

## References


[1] A. Goncalves, P. P. Ray, B. Soper, and R. Stevens, "Generation and evaluation of synthetic patient data," *Nature Communications*, vol. 11, no. 1, pp. 1–10, 2020.

[2] M. Nyström *et al.*, "Greater: Enhanced tabular data generation with language models," *arXiv preprint arXiv:2402.04015*, 2024.

[3] E. Kotelnikov, D. Baranchuk, I. Rubachev, and D. Vetrov, "Tabddpm: Modelling tabular data with diffusion models," in *Advances in Neural Information Processing Systems*, 2023.

[4] H. Zhang, J. Zhang, B. Srinivasan, Z. Shen, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," 2024. [Online]. Available: https://arxiv.org/abs/2310.09656

[5] S. Limnios, A. Elliott, M. Cucuringu, and G. Reinert, "Random walk based conditional generative model for temporal networks with attributes," *Preprint*, 2024, available from authors.

[6] M. Boedihardjo, T. Strohmer, and R. Vershynin, "Private measures, random walks, and synthetic data," 2024. [Online]. Available: https://arxiv.org/abs/2204.09167